

LOAD TESTING AND PIPELINES

AIM:

To create an Azure Load Testing resource and execute load testing for the Smart Event Management System and demonstrate CI/CD pipelines using Azure DevOps.

Load Testing :

Azure Load Testing is used to simulate multiple users accessing the application simultaneously in order to evaluate : Performance, Scalability, Reliability, Response time, Resource utilization
It helps identify bottlenecks before deployment.

Steps to Create an Azure Load Testing Resource:

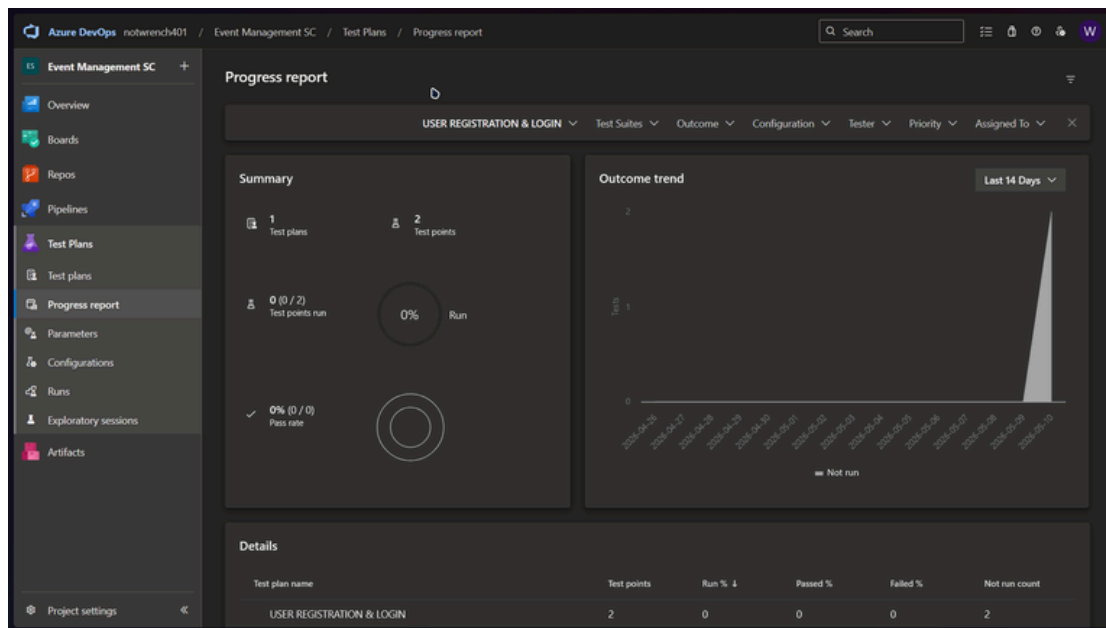
Before you run your first test, you need to create the Azure Load Testing resource:

1. Sign in to Azure Portal
Go to <https://portal.azure.com> and log in.
2. Create the Resource
 - o Go to *Create a resource* → Search for “Azure Load Testing”.
 - o Select Azure Load Testing and click Create.
3. Fill in the Configuration Details
 - o *Subscription*: Choose your Azure subscription.
 - o *Resource Group*: Create new or select an existing one.
 - o *Name*: Provide a unique name (no special characters).
 - o *Location*: Choose the region for hosting the resource.
4. Click Review + Create, then Create.
5. Once deployment is complete, click Go to resource.

Steps to Create and Run a Load Test:

Once your resource is ready:

1. Go to your Azure Load Testing resource and click Add HTTP requests > Create.
2. Basics Tab
 - o *Test Name*: Provide a unique name.
 - o *Description*: (Optional) Add test purpose.
 - o *Run After Creation*: Keep checked.
3. Load Settings
 - o *Test URL*: Enter the target endpoint (e.g., <https://yourapi.com/products>).
4. Click Review + Create → Create to start the test.



Pipelines

Description:

This experiment demonstrates how to integrate the Smart Event Management System with Azure DevOps pipelines. The pipeline automatically:

- Installs dependencies
- Builds the project
- Runs test cases
- Publishes artifacts
- Deploys application

This ensures continuous integration and reliable deployment.

Steps:

1. Connect GitHub to Azure DevOps:
 - o In Azure DevOps, create a new project.
 - o Create a pipeline and select GitHub as the source.
 - o Authorize access to your GitHub repository, ensuring that Azure DevOps can pull the repository for your pipeline.
2. Create azure-pipelines.yml File
 - o In your GitHub repository, create a new file called azure-pipelines.yml in the root directory.
 - o Add the following YAML configuration in repository root:

```
trigger:
- main

pool:
  vmImage: ubuntu-latest

steps:
- checkout: self

- task: NodeTool@0
  inputs:
    versionSpec: 'latest'
  displayName: "Set up Node.js"

- script: |
  curl -fsSL https://bun.sh/install | bash
  export BUN_INSTALL="$HOME/.bun"
  export PATH="$BUN_INSTALL/bin:$PATH"
  bun --version
  displayName: "Install Bun"

- script: |
  export BUN_INSTALL="$HOME/.bun"
  export PATH="$BUN_INSTALL/bin:$PATH"
  bun install
  displayName: "Install Dependencies"

- script: |
  export BUN_INSTALL="$HOME/.bun"
  export PATH="$BUN_INSTALL/bin:$PATH"
  bun test
  displayName: "Run Test Cases"

- script: |
  echo "Smart Event Management System Build Successful"
  displayName: "Verify Build"
```

Pipeline Tasks

- Runtime Environment Setup:
- Configures the Bun and Node.js runtime environment.
- Dependency Installation:
- Installs all required project dependencies using Bun.
- Automated Testing:
- Executes application test cases using Bun test runner.
- Build Verification:
- Confirms successful application build and pipeline execution.
- Deployment:
- Deploys the application to the configured hosting platform or cloud service.

RUN AND MONITOR PIPELINE:

1. Commit Changes

Push code to main branch.

2. Trigger Pipeline

Azure DevOps automatically starts pipeline.

3. Monitor Logs

Check build status

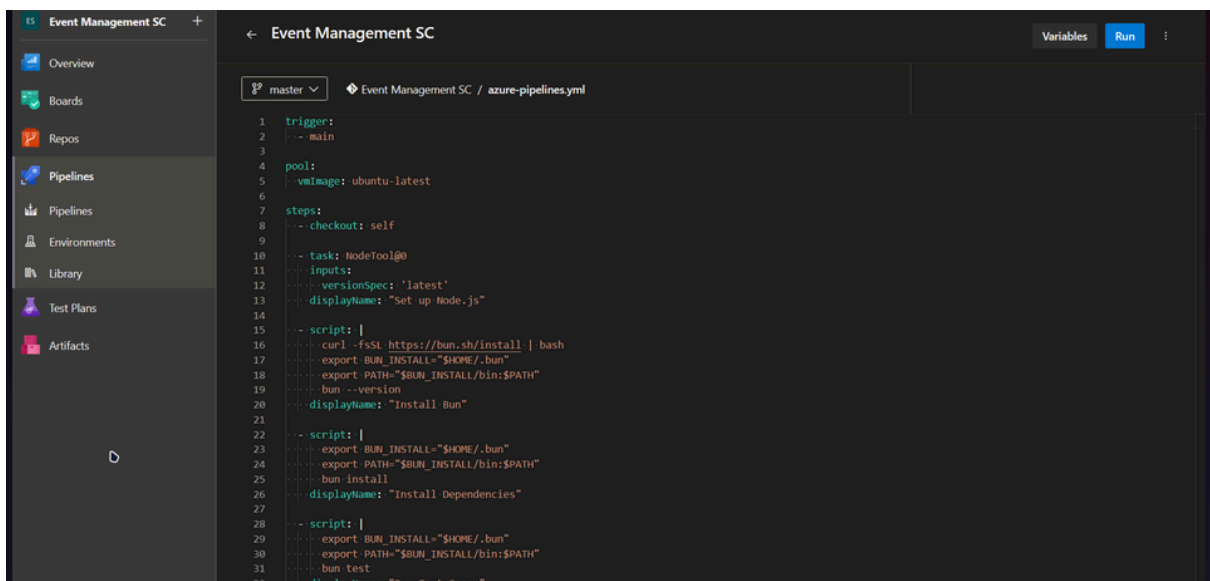
Verify test execution

Analyze errors if any

4. Verify Deployment

Confirm application deployment success.

Pipeline



The screenshot displays the Azure DevOps Pipelines interface. On the left, a sidebar shows navigation options: Overview, Boards, Repos, Pipelines (selected), Environments, Library, Test Plans, and Artifacts. The main area shows the 'Event Management SC' pipeline configuration for the 'master' branch, specifically the 'azure-pipelines.yml' file. The configuration is as follows:

```
1 trigger:
2   - main
3
4 pool:
5   vmImage: ubuntu-latest
6
7 steps:
8   - checkout: self
9
10  - task: NodeTool@0
11    inputs:
12      versionSpec: 'latest'
13      displayName: 'Set up Node.js'
14
15  - script: |
16    curl -fsSL https://bun.sh/install | bash
17    export BUN_INSTALL="$HOME/.bun"
18    export PATH="$BUN_INSTALL/bin:$PATH"
19    bun --version
20    displayName: 'Install Bun'
21
22  - script: |
23    export BUN_INSTALL="$HOME/.bun"
24    export PATH="$BUN_INSTALL/bin:$PATH"
25    bun install
26    displayName: 'Install Dependencies'
27
28  - script: |
29    export BUN_INSTALL="$HOME/.bun"
30    export PATH="$BUN_INSTALL/bin:$PATH"
31    bun test
32    displayName: 'Run Test Cases'
```

Result:

Successfully created Azure Load Testing resource and executed load testing for the Smart Event Management System. Successfully demonstrated Azure DevOps CI/CD pipeline with automated build, testing, and deployment.